NumPy — The High-Performance Math Engine

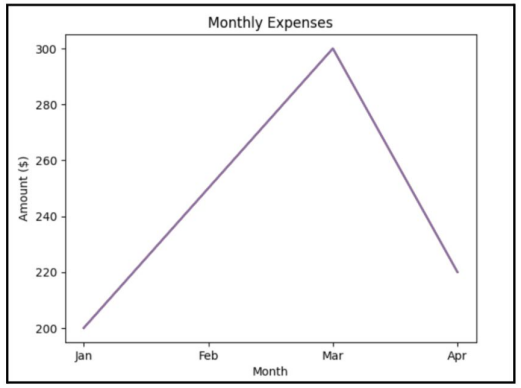
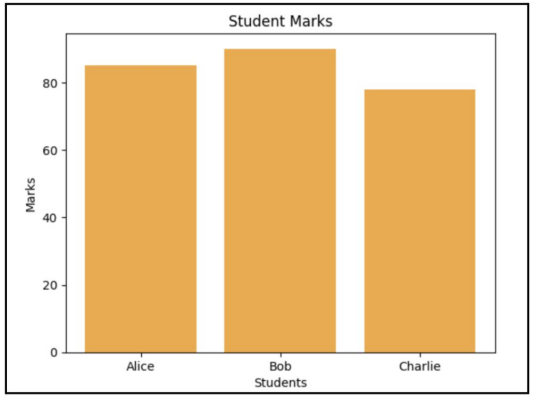
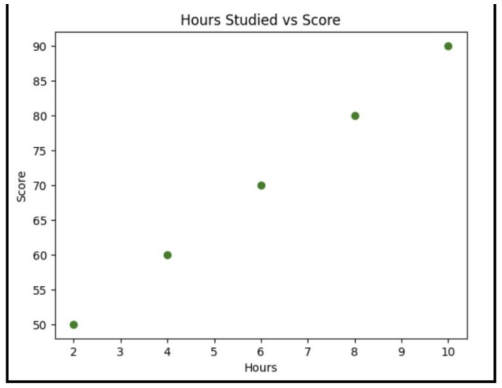
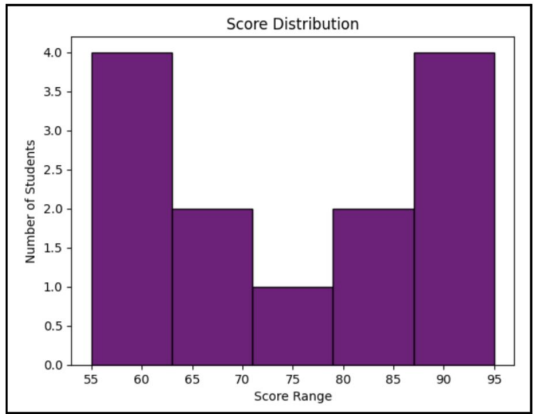
- NumPy: Foundation for data science library (like Pandas and TensorFlow).
  - Faster than python lists
  - Handles 1D, 2D and 3D Arrays.
- Memory & Speed Efficiency:
  - Python lists - store mixed data types with high overhead,
  - NumPy arrays (ndarrays) store single-type data in continuous memory blocks.
- Vectorization & Math: perform arithmetic across entire arrays instantly
  - Eliminating the need for slow for loops.
- The Power of Broadcasting: Math operations across arrays of different sizes
  - ex: Seamlessly adding a 1D vector to a 2D matrix.
- Getting Started: Open your terminal and run “pip install numpy pandas matplotlib“ to install the core data stack.

Pandas — Structuring & Cleaning Real-World Data

- DataFrame: A 2D table with rows (Axis 0) and columns (Axis 1), like an Excel spreadsheet.
- Series: A single column of a DataFrame.
- Data Exploration:
  - .head() – first rows.
  - .info() - data types and missing values.
  - .describe() - summaries like mean, min, and max.
- Data Selection:
  - .loc[] for label-based selection (e.g., finding a student by their name).
  - iloc[] for index-based selection (e.g., finding a student by their exact row number).
- Data Cleaning :
  - .isnull() - identifies missing values.
  - .fillna() - fills values.
  - astype() - fixes data types.
  - .drop_duplicates() - removes duplicates.

Pandas — Modifying, Aggregating, & Combining Data

- Modifying Columns: df["bonus"] = df["score"] + 5
  - .apply() to run custom functions on columns.
  - .str accessor for text .str.upper() or .str.contains().
- Grouping & Aggregating: summarize big datasets into insights.
  - .groupby() paired with .mean(), .sum(), or .count()
- Combining Datasets:
  - .merge(): Brings tables together based on a common identifier
  - expanding the data horizontally (adding columns).
  - .concat(): Stacks datasets on top of each other
  - expanding the data vertically (adding rows).
- Exporting: Save your cleaned and processed DataFrames
  - .to_csv()
  - .to_excel().

Matplotlib — Bringing Data to Life

- Choosing the Right Plot:
  - Line Plot (plt.plot): Tracking trends over time (e.g., monthly expenses).
  - Bar Plot (plt.bar): Comparing different categories (e.g., student marks).
  - Scatter Plot (plt.scatter): Identifying relationships between two variables (e.g., hours studied vs. exam score).
  - Histogram (plt.hist): Viewing the distribution and clustering of a dataset.
- Advanced Professional Features:
  - Build complex dashboards with multiple side-by-side subplots().
  - Array Visualization: Use imshow() to render 2D NumPy arrays directly as visual grids or images.